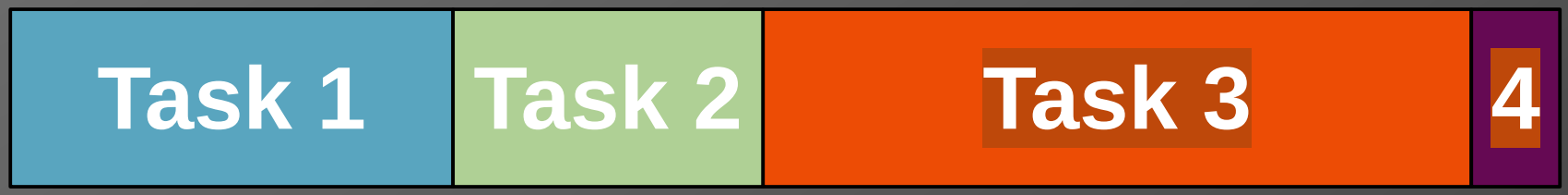
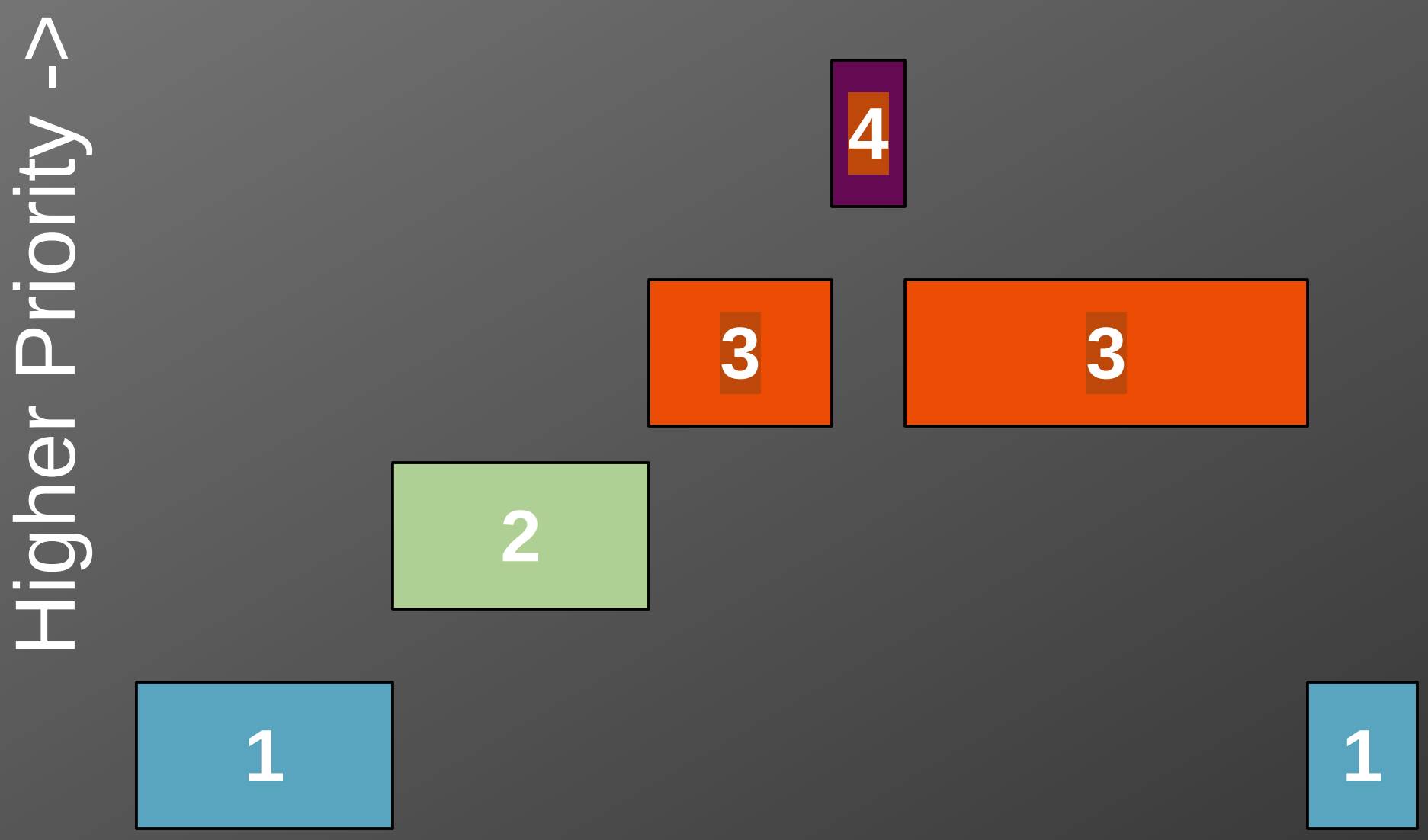
Multitasking on Microcontrollers

March 7, 2026

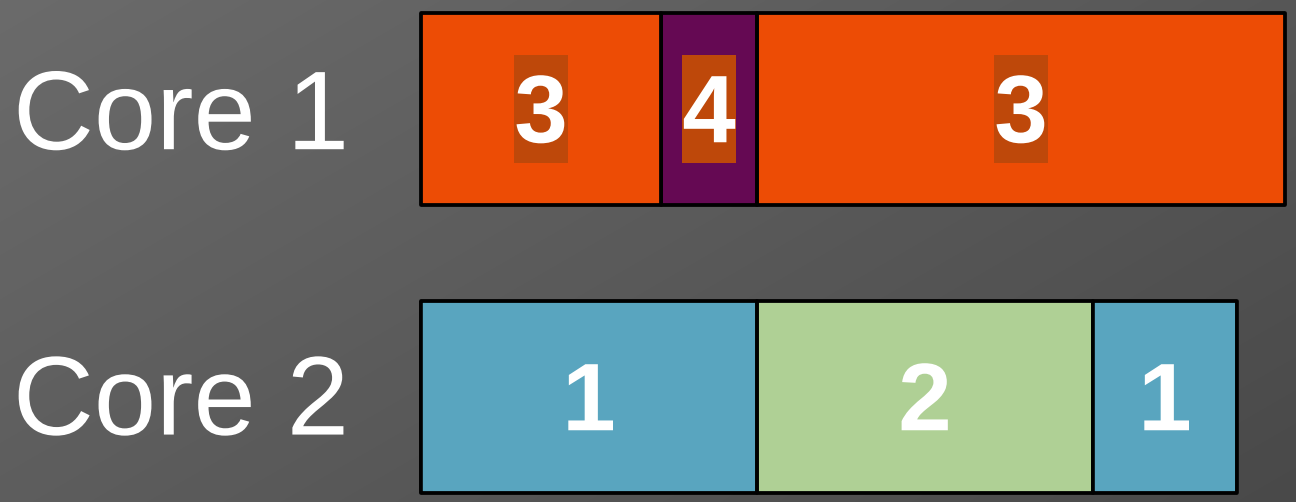
“Round Robin” Cooperative Multitasking



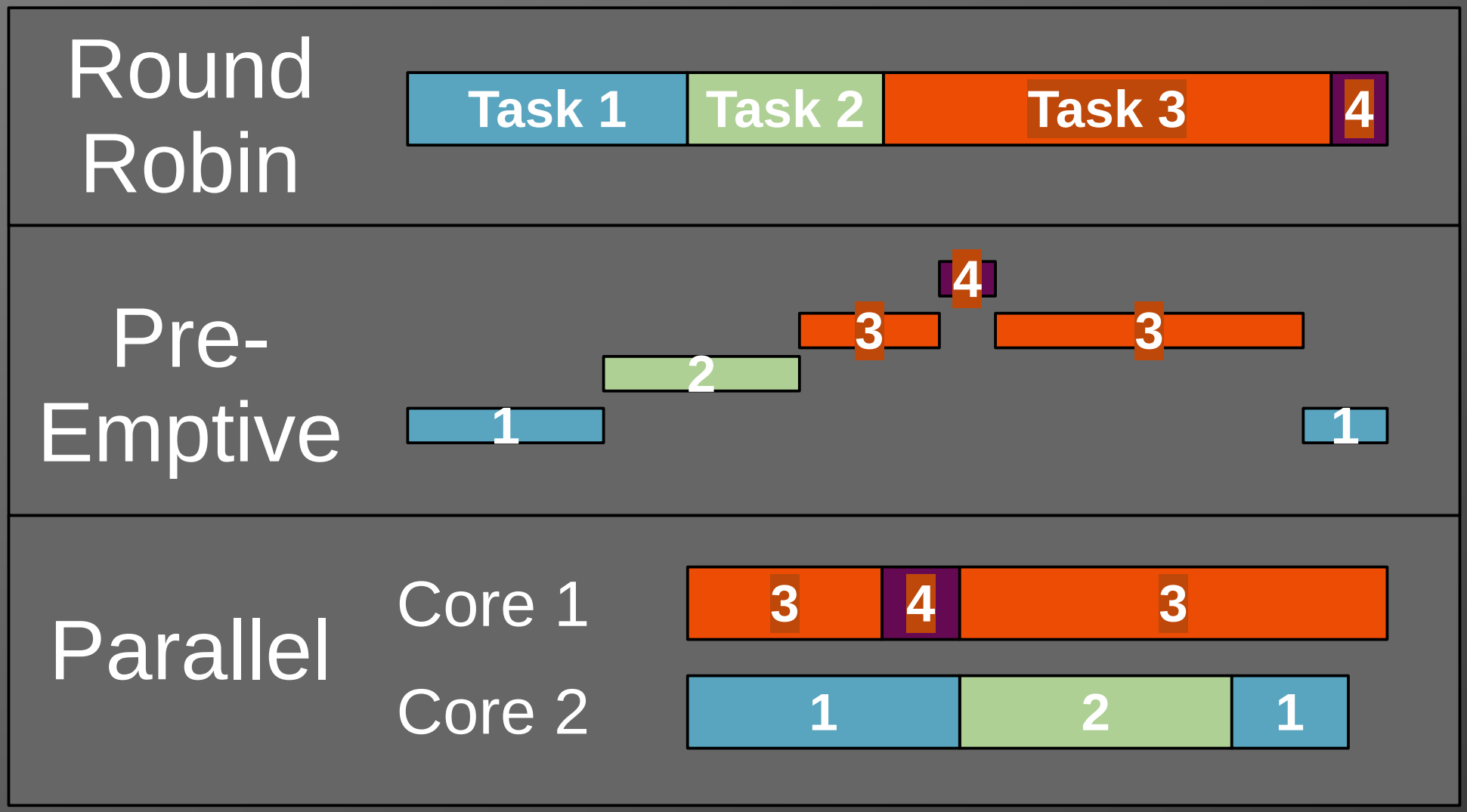
“Pre-emptive” Multitasking



Parallel Processing

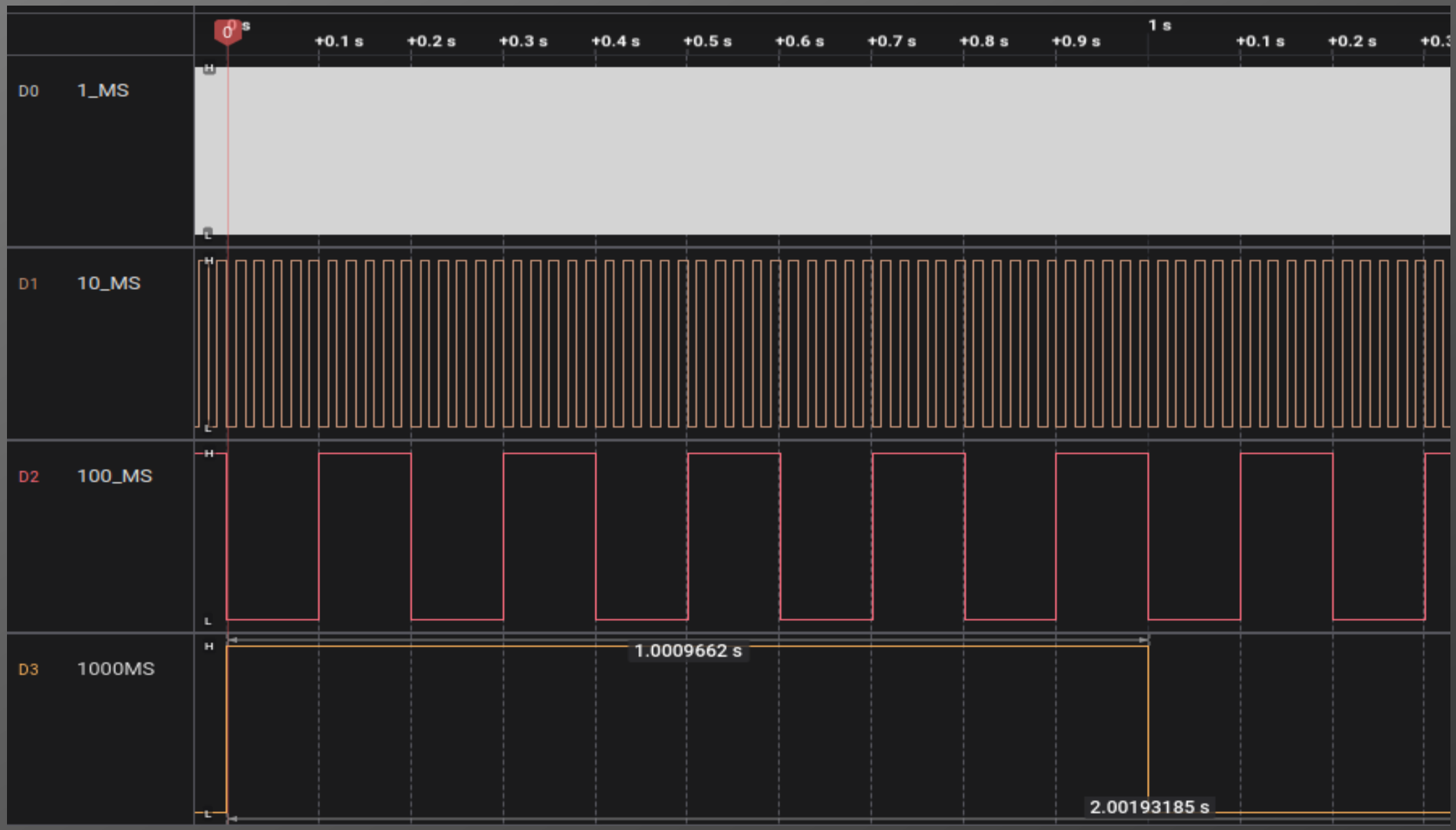


Trade-Offs?



Parallel: Mutex, Semaphore, Atomic Operation, Spinlock

MT1.ino: Round Robin Example



1 second = 1.009662 seconds

MT1 Code

```
const int TICK_INTERVAL = 1000; // 1000us is our 'tick'

const int LED1_10_MS   = 10;  // toggles every 10ms
const int LED1_100_MS  = 11;  // toggles every 100ms
const int LED1_1000_MS = 12;  // toggles every 1s
const int LED1_3000_MS = 13;  // toggles every 5s

bool led_10_ms_state   = LOW;
bool led_100_ms_state  = LOW;
bool led_1000_ms_state = LOW;
bool led_3000_ms_state = LOW;

unsigned long millisecondCounter = 0; // 0.001 seconds
unsigned long centisecondCounter = 0; // 0.01  seconds
unsigned long decisecondCounter  = 0;  // 0.1   seconds
unsigned long secondCounter      = 0;  // 1.0   seconds
unsigned long threeSecondCounter = 0;  // 3.0   seconds
```

```
unsigned long previousMicros = micros();

void loop() {
    unsigned long currentMicros = micros();

    // this block executes every 1ms
    if (currentMicros - previousMicros >= TICK_INTERVAL) {
        previousMicros = currentMicros;
        millisecondCounter++;

        // this block executes every 10ms
        if (millisecondCounter >= 10) {
            millisecondCounter = 0;
            centisecondCounter++;

            led_10_ms_state = !led_10_ms_state;
            digitalWrite(LED1_10_MS, led_10_ms_state);

            // this block executes every 100ms
            if (centisecondCounter >= 10) {
                centisecondCounter = 0;
                decisecondCounter++;

                led_100_ms_state = !led_100_ms_state;
                digitalWrite(LED1_100_MS, led_100_ms_state);

                if (decisecondCounter >= 10) {
                    decisecondCounter = 0;
                    secondCounter++;
```

MT2.ino: Multirate Blinker

```
const int ledPins[] = {10, 11, 12, 13};
const unsigned long intervals[] = {100000, 250000, 400000, 500000}; // in microseconds

// State variables
unsigned long lastToggles[] = {0, 0, 0, 0};
bool ledStates[] = {LOW, LOW, LOW, LOW};

const int numLeds = 4;
```

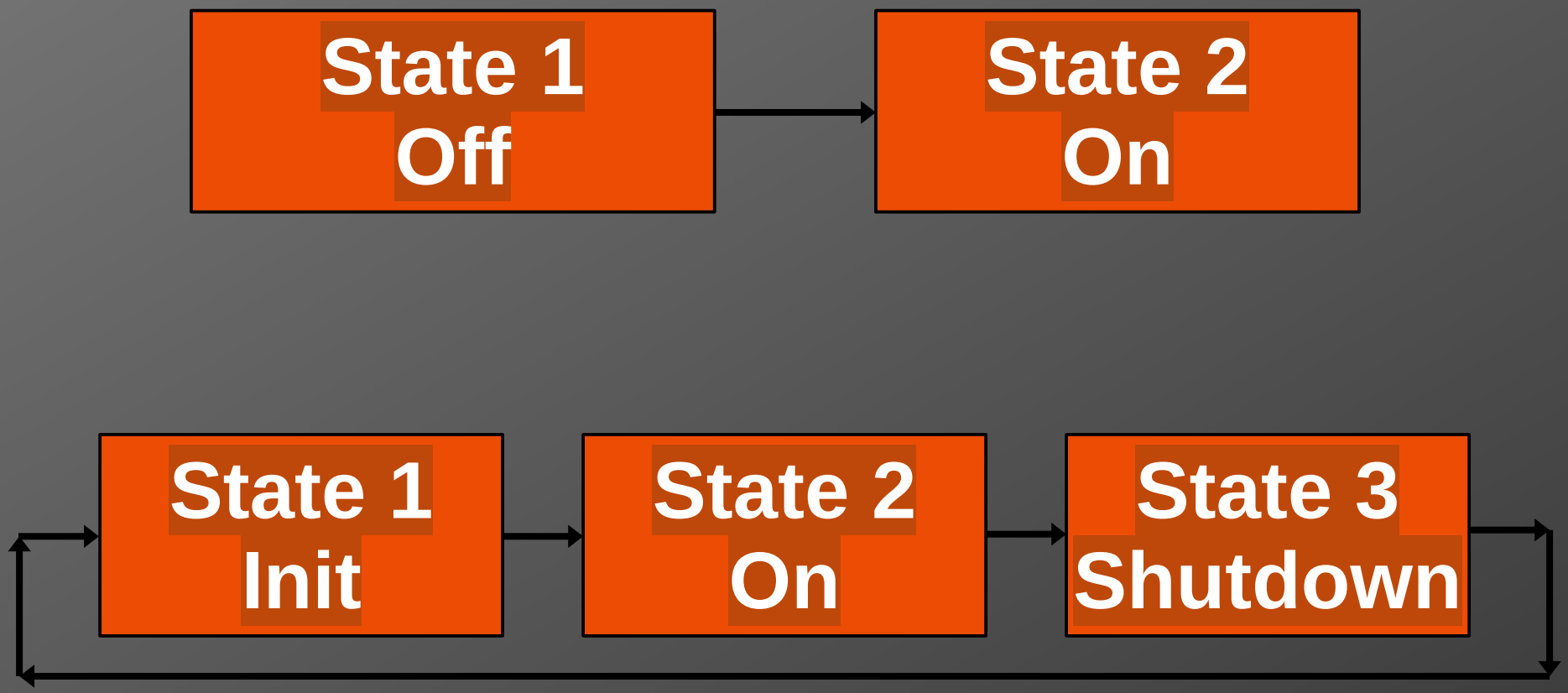
```
void loop() {
  unsigned long currentMicros = micros();

  // Task: Process LED 0
  if (currentMicros - lastToggles[0] >= intervals[0]) {
    lastToggles[0] = currentMicros;
    ledStates[0] = !ledStates[0];
    digitalWrite(ledPins[0], ledStates[0]);
  }

  // Task: Process LED 1
  if (currentMicros - lastToggles[1] >= intervals[1]) {
    lastToggles[1] = currentMicros;
    ledStates[1] = !ledStates[1];
    digitalWrite(ledPins[1], ledStates[1]);
  }

  // Task: Process LED 2
```

MT3.ino: Using struct tasks

```
// Structure to define our LED tasks
struct LedTask {
  int pin;
  unsigned long interval;    // Interval in microseconds
  unsigned long lastToggle;  // Last time the LED was flipped
  bool state;                // Current ON/OFF state
};

LedTask tasks[] = {
  {10, 100000, 0, LOW},  // LED 0: 10Hz (fast)
  {11, 250000, 0, LOW},  // LED 1: 4Hz
  {12, 500000, 0, LOW},  // LED 2: 2Hz
  {13, 1000000, 0, LOW}  // LED 3: 1Hz (slow)
};

const int numTasks = sizeof(tasks) / sizeof(tasks[0]);
```

```
void loop() {
  // Capture current time once per loop for consistency
  unsigned long currentMicros = micros();

  for (int i = 0; i < numTasks; i++) {
    if (currentMicros - tasks[i].lastToggle >= tasks[i].interval) {
      tasks[i].lastToggle = currentMicros;
      tasks[i].state = !tasks[i].state;
      digitalWrite(tasks[i].pin, tasks[i].state);
    }
  }
}
```

MT4.ino: State Machine



MT4.ino: State Machines

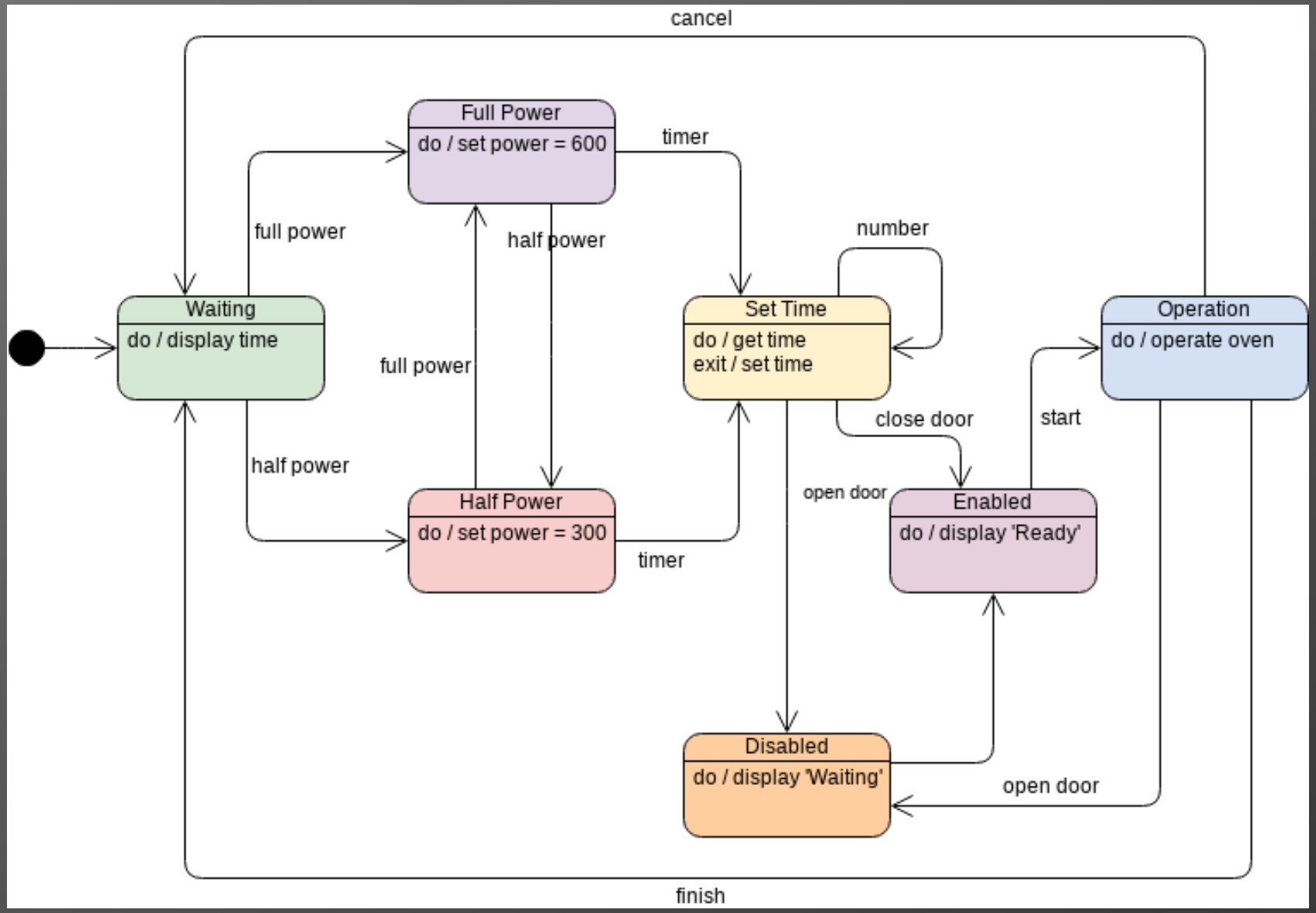
This is a very powerful cooperative multitasking technique for intermediate coders

Go from “writing a story” to “drawing a map”

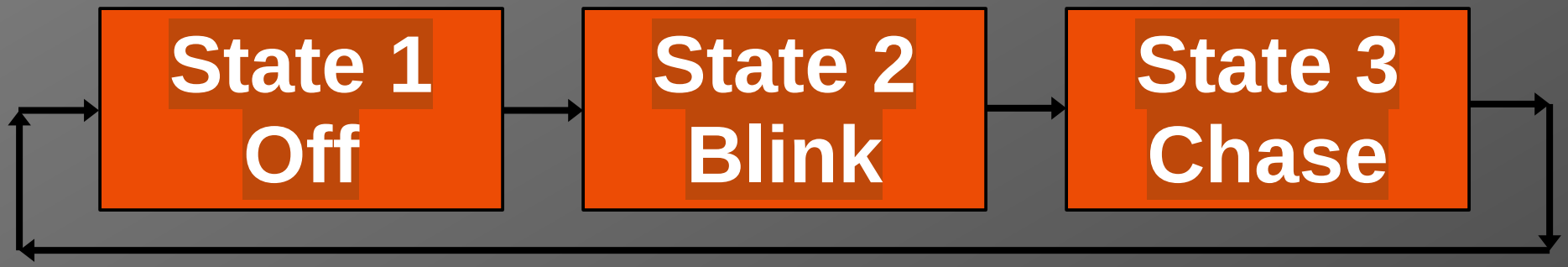
Pillars of FSM:

- States
- Events
- Transitions

MT4.ino: Microwave Oven State Machine



MT4.ino: LED State Machine




```
// State Machine Modes
enum SystemMode { ALL_OFF, INDEPENDENT_BLINK, CHASER };
SystemMode currentMode = ALL_OFF;
```

```
    // Detect Press (Falling Edge)
    if (currentButtonState == LOW && lastButtonState == HIGH) {
      // Cycle through states: 0 -> 1 -> 2 -> 0...
      currentMode = (SystemMode)((currentMode + 1) % 3);
```

```
  // Execute Logic based on the State Machine
  switch (currentMode) {

    case ALL_OFF:
      // LEDs stay off, nothing to do here
      break;

    case INDEPENDENT_BLINK:
      // Each LED blinks at a different rate (100ms,
```

MT4.ino: LED State Machine

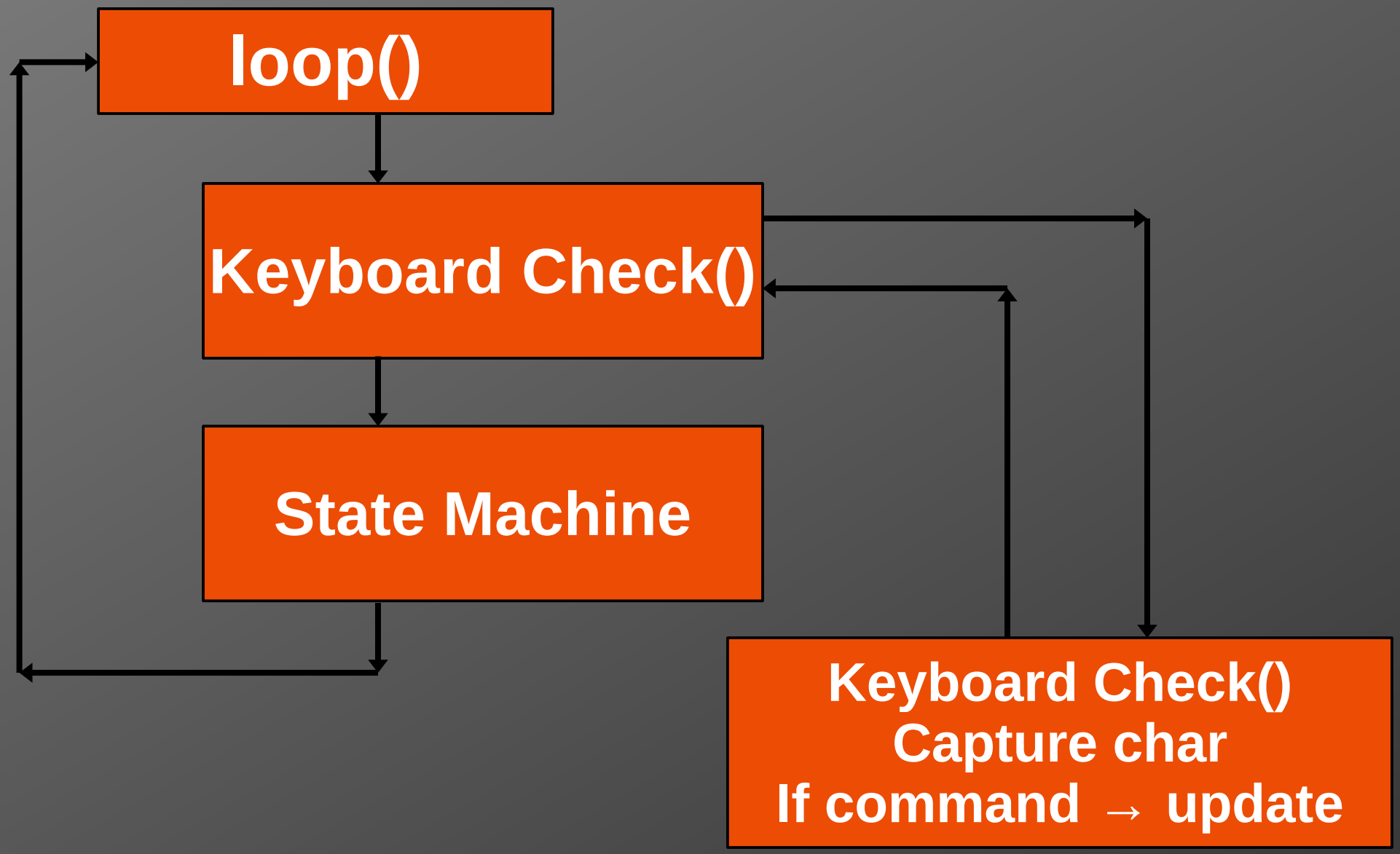
<u>Class exercise:</u>

Can you add a state where all LEDs are lit all the time?

```
// State Machine Modes
enum SystemMode { ALL_OFF, INDEPENDENT_BLINK, CHASER };
```

```
  // Execute Logic based on the State Machine
  switch (currentMode) {

    case ALL_OFF:
      // LEDs stay off, nothing to do here
      break;

    case INDEPENDENT_BLINK:
      // Each LED blinks at a different rate (100ms,
```

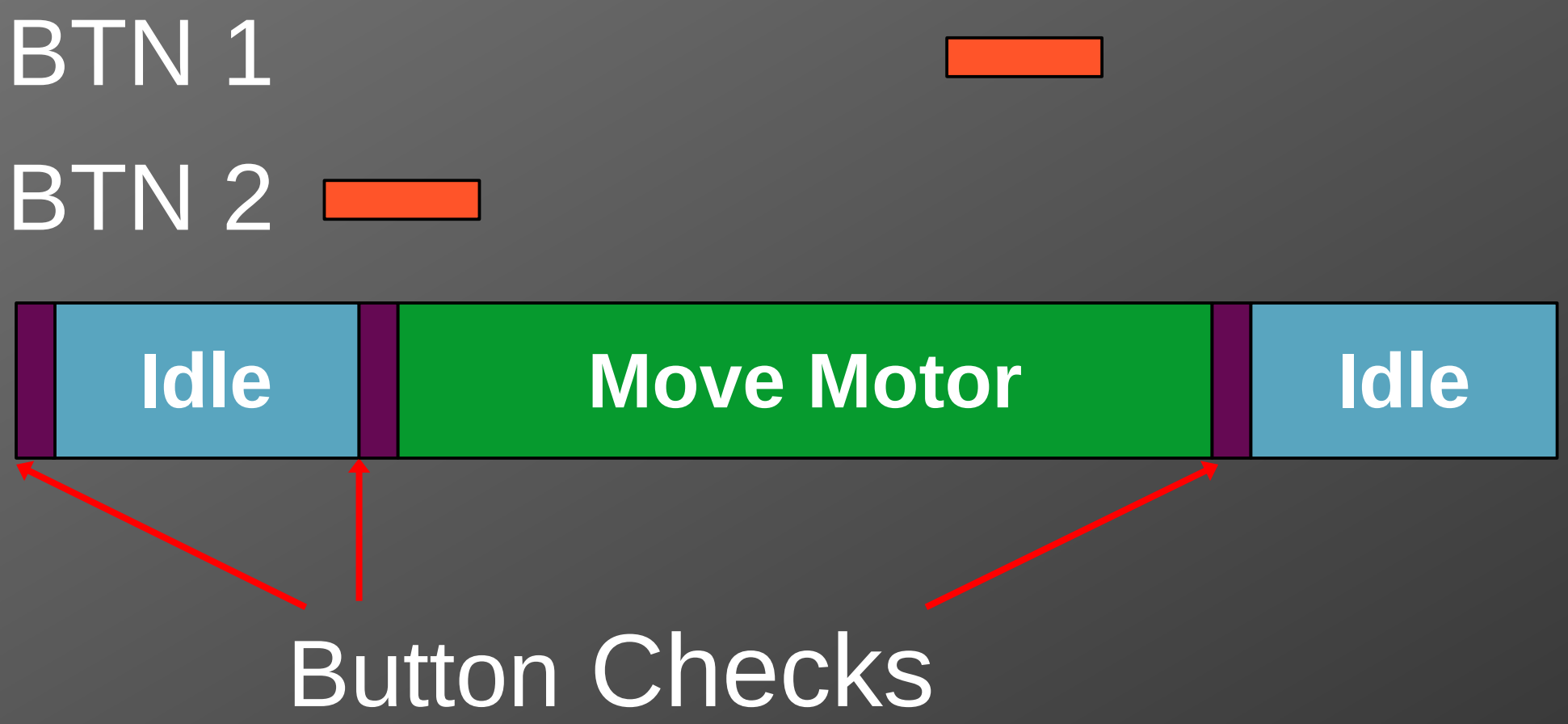
MT5.ino: Capture Keyboard



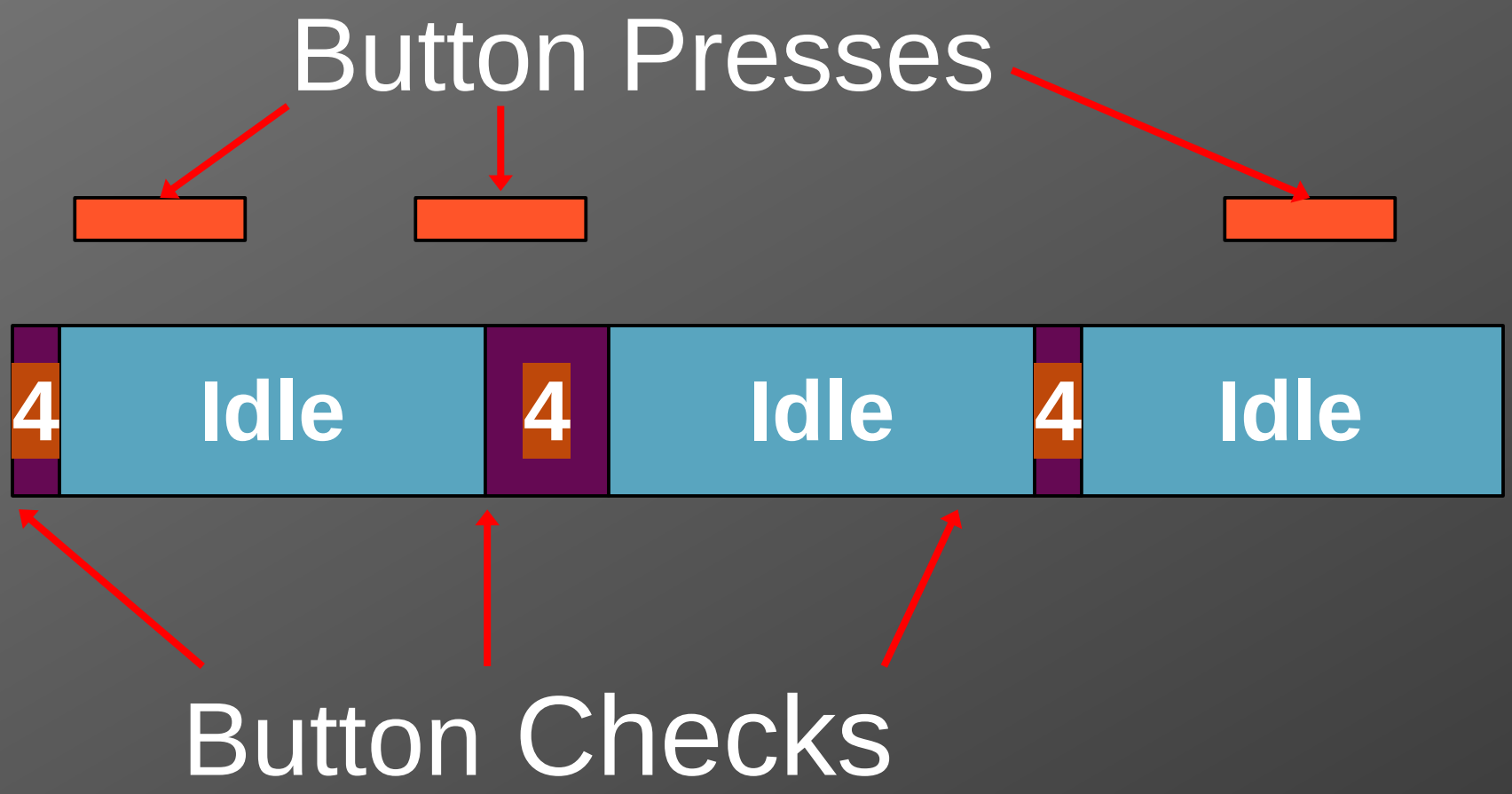
MT6.ino: Mediocre Button Press Counter

Btn Press Sequence	counter	binary
Initialized	0	0001
#1	1	0010
#2	2	0011
#3	3	0100
#4	4	0101
#5	5	0110
#6	6	0111
#7	7	1000
… #15	15	1111
#16	0	0000

MT6.ino: Mediocre Button Press Counter



MT6.ino: Mediocre Button Press Counter



Hint: Use hardware interrupt!